

Command Line Basics

A practical reference for the terminal (bash and zsh) on macOS and Linux. On Windows, use Git Bash and everything here works the same way.

1. How Commands Work

The terminal is a text interface to your computer. You type a command, press Enter, and read the result. Most commands follow one pattern:

```
command  -options  arguments
ls       -lh      data/          # "list the data folder, long human-readable format"
```

man <code>ls</code>	Open the manual page for any command (here, <code>ls</code>). Scroll with arrow keys, quit with <code>q</code> .
clear	Clear the screen. (Shortcut: <code>Ctrl+L</code>)
Ctrl+C	Cancel a running command and get your prompt back.

The two biggest time-savers: press **Tab** to auto-complete file and folder names (press it twice to see all options), and press the **Up arrow** to bring back previous commands so you never retype them.

2. Where Am I? Moving Around

pwd	Print working directory: show the folder you are currently in. Every command runs relative to this location.
cd <code>data</code>	Change directory into the <code>data</code> folder (inside the current folder).
cd <code>data/raw</code>	Go down two levels in one step.
cd <code>..</code>	Go up to the parent folder. <code>cd ../../</code> goes up two levels.
cd <code>~</code>	Jump to your home folder from anywhere. <code>cd</code> alone does the same.
cd <code>-</code>	Jump back to the folder you were in previously.
cd <code>/Users/ana/projects</code>	An absolute path starts with <code>/</code> and works from anywhere. A relative path (like <code>data/raw</code>) starts from where you are now.

Spaces in names: a folder called `My Drive` must be quoted (`cd "My Drive"`) or escaped (`cd My\ Drive`). Tab completion handles this for you. You can also drag a folder from Finder into the terminal to paste its full path.

3. Listing Files with `ls`

ls	List the files and folders in the current directory.
ls <code>-l</code>	Long format: one line per file with permissions, owner, size (in bytes), and last-modified date.
ls <code>-lh</code>	Long format with human-readable sizes (<code>4.2K</code> , <code>13M</code> , <code>2.1G</code>). The combination you will use most.
ls <code>-a</code>	Include hidden files (names starting with <code>.</code> , like <code>.gitignore</code>). Combine: <code>ls -lha</code> .
ls <code>-lt</code>	Sort by modification time, newest first. Great for finding what just changed.
ls <code>data/</code>	List the contents of another folder without moving into it.
ls <code>*.csv</code>	List only files matching a pattern (see wildcards below).

4. File and Folder Sizes

<code>ls -lh</code>	Shows the size of each file . Note: for folders it shows only the folder entry itself, not the contents.
<code>du -sh data/</code>	Disk usage of a folder including everything inside it: <code>-s</code> summarizes to one total, <code>-h</code> makes it human-readable.
<code>du -sh *</code>	Size of every item in the current directory, one line each.
<code>du -sh * sort -h</code>	The same, sorted from smallest to largest. Perfect for finding what is eating your disk space.

5. Wildcards (Glob Patterns)

Wildcards let one command act on many files at once. They work with `ls`, `cp`, `mv`, `rm`, and most other commands.

<code>*</code>	Matches any number of characters. <code>ls *.csv</code> lists every CSV file; <code>ls Day05*</code> lists everything starting with "Day05".
<code>?</code>	Matches exactly one character. <code>ls sales202?.csv</code> matches <code>sales2025.csv</code> , <code>sales2026.csv</code> , ...
<code>[...]</code>	Matches one character from a set. <code>ls report[12].txt</code> matches <code>report1.txt</code> and <code>report2.txt</code> .

Preview first: before running `rm *.csv` or `mv *.csv data/`, run `ls *.csv` with the same pattern to see exactly which files will be affected.

6. Creating, Copying, Moving, Renaming

<code>mkdir results</code>	Make a new directory called <code>results</code> .
<code>mkdir -p data/raw/2026</code>	Create nested folders in one step (<code>-p</code> creates the parents as needed).
<code>cp sales.csv backup.csv</code>	Copy a file to a new name. The original stays where it is.
<code>cp sales.csv data/</code>	Copy a file into a folder, keeping its name. <code>cp *.csv data/</code> copies all CSVs.
<code>cp -r project/ project-backup/</code>	Copy a whole folder and its contents (<code>-r</code> = recursive ; required for folders).
<code>mv sales.csv data/</code>	Move a file into a folder (cut and paste).
<code>mv draft.txt final.txt</code>	Renaming is just moving to a new name in the same place.
<code>mv sales.csv data/sales-2026.csv</code>	Move and rename in one step.

Careful: `cp` and `mv` silently overwrite an existing file with the same name. Add `-i` (`cp -i`, `mv -i`) to be asked before overwriting.

7. Deleting Files and Folders

<code>rm old_notes.txt</code>	Remove a file. <code>rm *.tmp</code> removes every file matching the pattern.
<code>rm -i old_notes.txt</code>	Ask for confirmation before each deletion. A good habit while learning.
<code>rmdir temp/</code>	Remove an empty folder. Fails (safely) if anything is inside.
<code>rm -r temp/</code>	Remove a folder and everything inside it , recursively.

There is no Trash on the command line. `rm` deletes permanently: no confirmation, no undo. Double-check the name (or preview with `ls`) before pressing Enter, especially with wildcards or `-r`.

8. Viewing File Contents

<code>cat README.md</code>	Print the whole file to the screen. Best for short files.
<code>head sales.csv</code>	Show the first 10 lines: a quick peek at a file's structure and header row.
<code>head -n 3 sales.csv</code>	Show the first 3 lines (<code>-n</code> sets the number of lines).
<code>tail sales.csv</code>	Show the last 10 lines. <code>tail -n 20</code> shows the last 20.
<code>less big_file.csv</code>	View a large file one page at a time. Scroll with arrows or space, search with <code>/</code> , quit with <code>q</code> .
<code>wc -l sales.csv</code>	Word count's <code>-l</code> option counts lines: how many rows are in this file?

Why this matters for analytics: `head data.csv` shows you the column names and layout of a giant CSV in a fraction of a second, before you ever load it into Python or Excel.

9. Combining Commands

<code>cmd1 cmd2</code>	The pipe sends the output of one command into the next as its input.
<code>ls -lt head -n 5</code>	The 5 most recently modified files.
<code>wc -l *.csv sort -n</code>	Line counts of every CSV, sorted numerically.
<code>head -n 100 big.csv > sample.csv</code>	<code>></code> redirects output into a file instead of the screen (creating or overwriting it).
<code>echo "note" >> log.txt</code>	<code>>></code> appends to the end of a file instead of overwriting it.
<code>mkdir out && cd out</code>	<code>&&</code> runs the second command only if the first one succeeded.

10. A Few More Essentials

<code>grep "Boston" sales.csv</code>	Print every line in the file containing the text "Boston". Add <code>-i</code> to ignore case, <code>-c</code> to just count matches.
<code>grep -r "TODO" .</code>	Search for text in every file in the current folder and all subfolders.
<code>find . -name "*.ipynb"</code>	Find files by name anywhere under the current folder.
<code>history</code>	Show your previous commands. <code>history grep git</code> finds the git ones.
<code>open .</code> (macOS)	Open the current folder in Finder. On Windows Git Bash: <code>explorer .</code>
<code>which python</code>	Show the full path of the program a command actually runs.

MSBA Bootcamp · Day 5 · Command Line Basics — keep this next to your keyboard until `cd`, `ls -lh`, and Tab completion are muscle memory.